

LAPORAN FINAL PROJECT DEEP LEARNING
REAL LIFE VIOLENCE DETECTION



Penyusun

Valentino Reswara Ajiputra - 5025221017

Rafli Syahputra Pane - 5025221038

Naufal Khairul Rizky - 5025221127

Ivan Fairuz Adinata - 5025221167

DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA

2025

DAFTAR ISI

BAB 1 PENDAHULUAN	2
1.1. Latar Belakang	2
1.2. Rumusan Masalah	2
1.3. Batasan Masalah	2
2. BAB 2 TINJAUAN PUSTAKA	4
2.1. Penelitian Terkait	4
2.2. Dasar Teori	5
2.2.1. MobileNet	5
2.2.2. EfficientNet	6
2.2.3. BiLSTM	7
2.2.4. TimeSformer	9
3. BAB 3 METODOLOGI	11
3.1. Dataset yang Digunakan	11
3.2. Desain Sistem	12
3.2.1. Tahap Preprocessing Data	12
3.2.2. Tahap Training Model	13
3.2.2.1. MobileNet + BiLSTM	13
3.2.2.2. EfficientNet + BiLSTM	15
3.2.2.3. TimeSformer	16
3.2.3. Tahap Testing dan Evaluasi	17
4. BAB 4 HASIL & PEMBAHASAN	18
4.1. Skenario Pengujian	18
4.1.1. Penggunaan SGD Optimizer	18
4.1.2. Penggunaan Adam Optimizer	18
4.1.3. Penambahan Data dari Sumber Baru	19
4.2. Pembahasan	19
5. BAB 5 KESIMPULAN DAN SARAN	21
5.1. Kesimpulan	21
5.2. Saran	21
6. DAFTAR PUSTAKA	22

BAB 1 PENDAHULUAN

1.1. Latar Belakang

Kehidupan sosial bergantung pada aturan hukum dan keamanan kota, dengan sistem pengawasan memainkan peran penting dalam menangani insiden kekerasan. Namun sistem alarm manual sering kali menghadapi *delay* dan ketidakpastian, serta pengawasan oleh manusia saja terbukti kurang efisien. Dalam konteks ini, teknologi kecerdasan buatan dan pengolahan video bisa menjadi solusi untuk permasalahan-permasalahan tersebut melalui implementasi model deteksi kekerasan otomatis berbasis *deep learning*.

Mengatasi masalah tersebut, kami menggunakan model deep learning seperti MobileNet, EfficientNet. Model-model tersebut telah banyak digunakan dalam klasifikasi citra maupun video karena efisiensi dan performanya yang tinggi. Kombinasi model visual ini dengan arsitektur BiLSTM dapat memanfaatkan informasi spasial dan temporal secara bersamaan, yang sangat penting dalam memahami konteks kekerasan dalam video. Model berbasis *transformer* yang menggunakan mekanisme *self-attention* seperti TimeSformer juga akan digunakan untuk mengatasi masalah ini.

Final project ini bertujuan untuk membangun sistem *Real Life Violence Detection* berbasis *deep learning*, dengan membandingkan kinerja tiga kombinasi model: MobileNet + BiLSTM, EfficientNet + BiLSTM, dan TimeSFormer. Untuk mendapatkan hasil yang optimal, dilakukan pengujian terhadap beberapa skenario, termasuk penggantian optimizer dan penambahan data pelatihan.

1.2. Rumusan Masalah

Rumusan masalah dalam final project ini adalah sebagai berikut:

1. Bagaimana performa akurasi serta *training time* dari model MobileNet + BiLSTM, EfficientNet + BiLSTM, dan TimeSFormer dalam mendeteksi kekerasan dalam video?
2. Apakah perubahan optimizer dari *Stochastic Gradient Descent* (SGD) menjadi Adam dapat meningkatkan akurasi atau mengurangi *training time* dari ketiga model tersebut?
3. Bagaimana pengaruh penambahan dataset dari 2000 menjadi 4000 terhadap akurasi dan *training time* masing-masing model?

1.3. Batasan Masalah

Ruang lingkup penelitian pada final project ini dibatasi pada:

- Dataset yang digunakan terdiri dari 2000 video dari kaggle dan 2000 video tambahan dari Harvard University, total 4000 video.
- Model yang diuji terbatas pada kombinasi MobileNet + BiLSTM, EfficientNet + BiLSTM, dan TimeSFormer.
- Optimizer yang dibandingkan hanya SGD dan Adam.
- Evaluasi hanya difokuskan pada metrik akurasi serta *training time*.
- Implementasi dilakukan dalam lingkungan eksperimen dan belum diterapkan dalam sistem real-time.

1.4. Tujuan dan Manfaat

Tujuan:

- Membangun dan menguji performa tiga model deteksi kekerasan berbasis deep learning: MobileNet + BiLSTM, EfficientNet + BiLSTM, dan TimeSFormer.
- Melakukan evaluasi dampak penggantian optimizer terhadap performa model.
- Mengkaji pengaruh penambahan jumlah dataset terhadap peningkatan akurasi deteksi.

Manfaat:

- Memberikan referensi pemilihan model terbaik untuk deteksi kekerasan dalam video
- Menunjukkan pentingnya pemilihan optimizer dalam pelatihan model deep learning.
- Mendorong pengembangan sistem keamanan berbasis deep learning untuk mencegah tindak kekerasan di dunia nyata.

BAB 2 TINJAUAN PUSTAKA

2.1. Penelitian Terkait

Perkembangan pesat di bidang *computer vision* dan *deep learning* membuka peluang besar dalam menciptakan sistem deteksi kekerasan otomatis yang semakin canggih dan andal. Saat ini, fokus utama penelitian adalah deteksi kekerasan berbasis video karena potensinya dalam meningkatkan keamanan publik melalui sistem pengawasan yang lebih pintar dan responsif. Tantangan utama yang dihadapi adalah mengembangkan model yang tidak hanya akurat mengenali berbagai bentuk tindakan kekerasan dari berbagai konteks, tetapi juga efisien agar bisa dijalankan secara real-time. Berbagai pendekatan telah dicoba, mulai dari metode tradisional yang mengandalkan fitur manual hingga arsitektur deep learning modern seperti CNN dan RNN.

Salah satu penelitian terkini yang relevan adalah karya Fu et al. (2024) yang memperkenalkan model deteksi kekerasan berbasis kombinasi EfficientNet-B0 dan Separable Convolutional LSTM (SepConvLSTM) yang dilengkapi mekanisme self-attention. EfficientNet-B0 digunakan sebagai backbone untuk mengekstrak fitur spasial dari setiap frame, sementara SepConvLSTM memperkuat pemahaman model terhadap hubungan spasial-temporal antar frame. Uji coba pada dataset RWF2000, yang merupakan salah satu dataset video kekerasan terbesar, menunjukkan bahwa model ini mampu mencapai akurasi 90,75%. Temuan ini menegaskan bahwa kombinasi arsitektur ringan dan mekanisme self-attention sangat efektif untuk deteksi kekerasan berbasis video dalam skenario dunia nyata.

Penelitian lain dalam bidang deteksi kekerasan berbasis video adalah karya Vijeikis et al. (2022) yang mengusulkan arsitektur novel untuk deteksi kekerasan dari kamera pengawasan. Model yang diajukan mengombinasikan spatial feature extractor berbasis U-Net dengan MobileNet V2 sebagai encoder, diikuti dengan LSTM untuk ekstraksi fitur temporal dan klasifikasi. Keunggulan utama pendekatan ini terletak pada desainnya yang computationally light namun tetap mempertahankan performa yang baik. Eksperimen yang dilakukan pada dataset RWF-2000 menunjukkan bahwa model ini mencapai akurasi rata-rata 82.0% dan presisi rata-rata 81.2%.

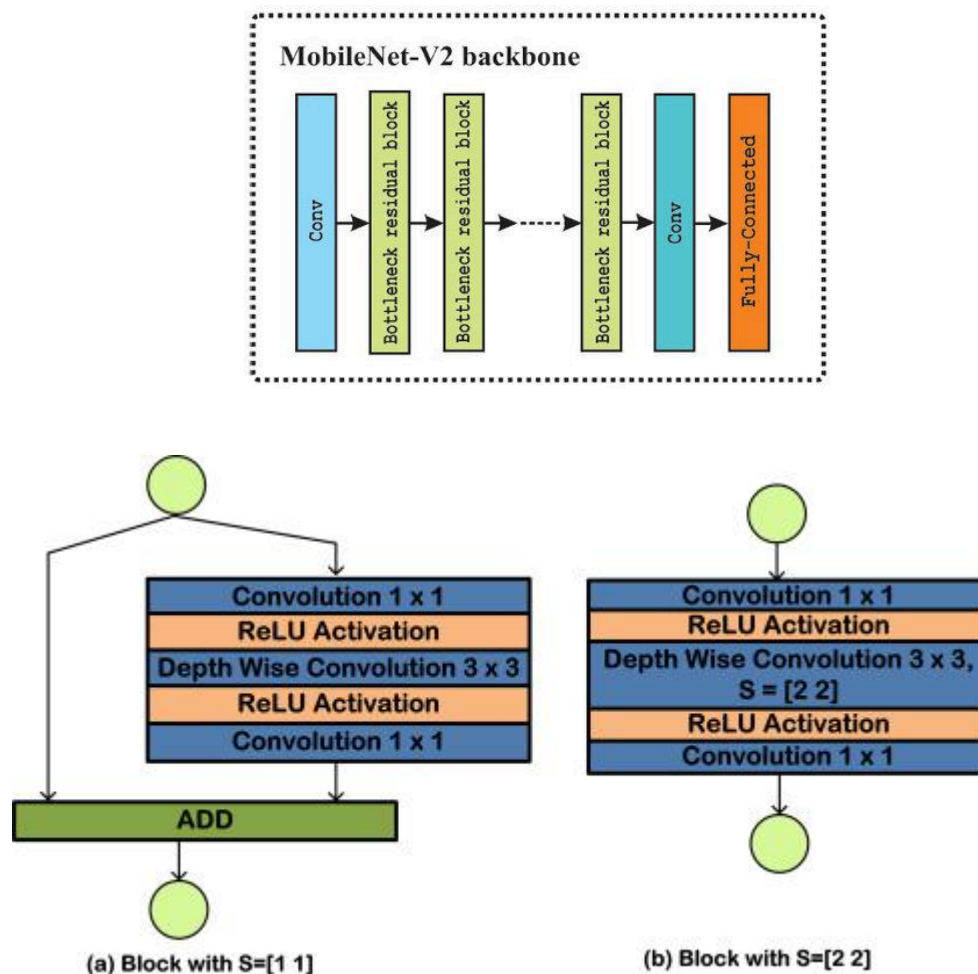
Penelitian oleh Vosta et al. (2022) mengusulkan model gabungan ResNet50 dan ConvLSTM untuk mendeteksi kejadian abnormal dalam video pengawasan, menggunakan dataset UCF-Crime. Model ini memanfaatkan ResNet50 untuk ekstraksi fitur spasial dan ConvLSTM untuk analisis temporal, dan berhasil mencapai AUC sebesar 81,71%, lebih tinggi dibandingkan pendekatan sebelumnya seperti C3D. Meski menunjukkan peningkatan performa, penelitian ini masih memiliki beberapa keterbatasan. Tantangan utama meliputi variasi pencahayaan, durasi kejadian abnormal yang sangat singkat, dan akurasi yang masih rendah dalam klasifikasi multi-kategori (14 kelas). Selain itu, karena UCF-Crime merupakan kumpulan video dunia nyata dengan latar belakang yang kompleks, model cenderung kesulitan dalam menggeneralisasi.

Dari berbagai studi tersebut, dapat disimpulkan bahwa deteksi kekerasan berbasis video memerlukan kombinasi arsitektur yang mampu menangkap fitur spasial dan temporal secara efisien. Meski demikian, belum ada penelitian yang secara komprehensif membandingkan efek optimizer (SGD vs Adam), generalisasi model pada dataset campuran (staged + real-world), dan penggunaan tiga model hybrid (MobileNet+BiLSTM, EfficientNet+BiLSTM, dan TimeSformer) pada *violence detection* berbasis video. Oleh karena itu, penelitian ini hadir untuk mengisi celah tersebut dengan melakukan evaluasi mendalam dalam berbagai skenario pengujian.

2.2. Dasar Teori

2.2.1. MobileNet

MobileNet merupakan arsitektur neural network yang telah dioptimalkan agar dapat digunakan secara efisien pada perangkat seluler. Dalam hal ini, digunakan model MobileNetV2 yang telah diseimbangkan ukuran model, efisiensi komputasi, dan akurasi sehingga lebih baik untuk klasifikasi sebuah gambar. MobileNet menggunakan depthwise separable convolution, sebuah teknik optimasi untuk mengurangi jumlah parameter dan biaya komputasi dari operasi konvolusi. Ide utama di balik teknik ini adalah untuk membagi operasi konvolusi tradisional menjadi dua proses yang terpisah: depthwise convolution dan pointwise convolution. (Garg et al., 2024).



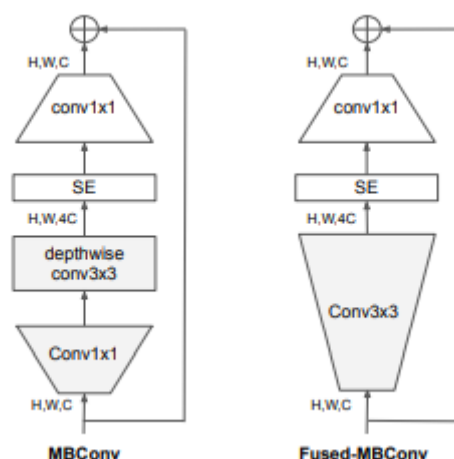
Gambar 1. Arsitektur MobileNet

Arsitektur utama dari *MobileNetV2* tersusun dari serangkaian blok yang bernama *Bottleneck Residual Block*. Dalam kerangka utamanya, *Bottleneck Residual Block* diawali dengan lapisan konvolusi dan diakhiri dengan lapisan konvolusi kembali lalu dilanjutkan dengan *fully connected layer*. Struktur nya biasa dibidang memiliki dua susunan, yakni *depth-wise convolution* dan *point-wise convolution*. *Depth-wise convolution* terdapat pada layer pertama, yaitu *convolution 1x1* dan blok 3x3 yang berfungsi melakukan filtering untuk mencari pola spasial. Sementara itu, *point-wise convolution* terdapat pada lapisan *convolution 1x1* setelah *depth-wise convolution* yang berfungsi menyusun fitur baru melalui kombinasi linier dari input channel. (Chandola et al., 2021)

Dalam arsitektur *MobileNetV2*, terdapat dua jenis *Bottleneck Residual Block* yang dibedakan oleh nilai stride-nya. Pada gambar (a) terdapat Block with Stride = [1 1] yang berfungsi untuk residual block. Di sini, dilakukan *feature extraction* tanpa mengubah ukuran spasial (tinggi dan lebar) dari *feature map*. Selanjutnya, pada gambar (b) terdapat Block with Stride = [2 2] sebagai downsizing block. Tujuan utamanya ialah melakukan *downsizing* atau mengecilkan ukuran *feature map*. (Chandola et al., 2021)

2.2.2. EfficientNet

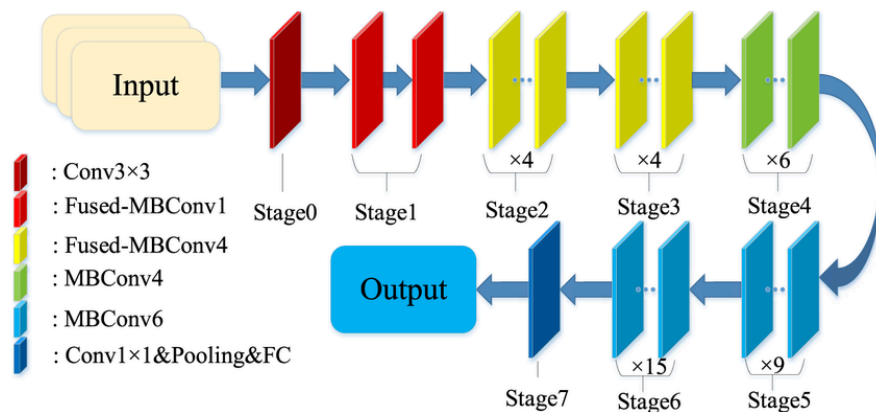
EfficientNet merupakan arsitektur CNN yang dirancang untuk efisiensi tinggi melalui pendekatan *compound scaling*, yaitu metode penskalaan terkoordinasi pada depth, width, dan resolution. EfficientNet dapat mencapai akurasi yang tinggi dengan jumlah parameter dan komputasi yang lebih rendah dari model-model seperti ResNet. Dalam project ini digunakan EfficientNetV2S, EfficientNetV2 merupakan pengembangan arsitektur model CNN dari EfficientNet, yang sebelumnya memperkenalkan *compound scaling* sebagai strategi penskalaan bersama pada depth, width, dan input resolution secara optimal (Tan et al., 2021).



Gambar 2. Perbedaan antara MBConv dengan Fused-MBConv.

Model EfficientNetV2 menggunakan pendekatan *progressive learning* dengan melatih model dengan resolusi rendah terlebih dahulu, kemudian perlahan-lahan meningkatkan resolusi selama pelatihan. Model ini menggunakan Fused-MBConv, menggantikan blok MBConv pada bagian awal jaringan pada EfficientNet. Fused-MBConv menggabungkan 3x3 convolution + batch normalization + activation tanpa menggunakan depthwise separable convolution pada blok awal seperti yang digunakan EfficientNet(Tan et al., 2021).

Stage	Operator	Stride	#Channels	#Layers
0	Conv3x3	2	24	1
1	Fused-MBConv1, k3x3	1	24	2
2	Fused-MBConv4, k3x3	2	48	4
3	Fused-MBConv4, k3x3	2	64	4
4	MBConv4, k3x3, SE0.25	2	128	6
5	MBConv6, k3x3, SE0.25	1	160	9
6	MBConv6, k3x3, SE0.25	2	256	15
7	Conv1x1 & Pooling & FC	-	1280	1



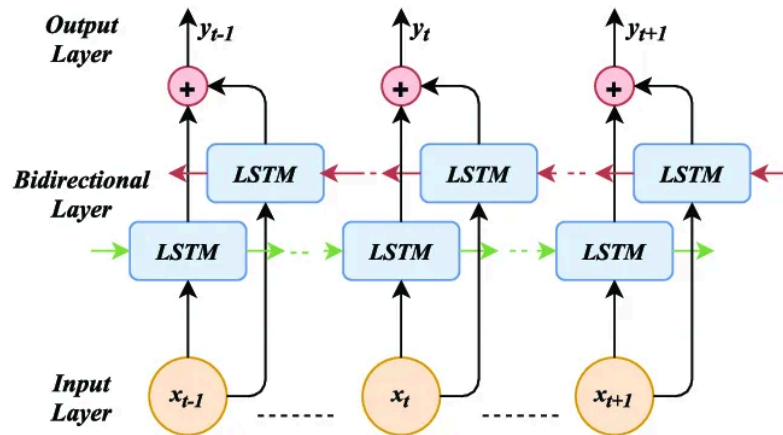
Gambar 3. EfficientNetV2 Architecture

Dalam klasifikasi video, model ini berfungsi sebagai feature extractor yang menerima input berupa frame dari video. Setiap frame akan diproses melalui lapisan awal berupa blok Fused-MBConv yang menggunakan convolutional standar untuk mengekstraksi fitur-fitur dasar. Selanjutnya, frame melewati beberapa blok MBConv yang terdapat depthwise separable convolution serta mekanisme squeeze-and-excitation(SE) untuk menangkap fitur spasial dan channel-wise yang lebih kompleks. Setelah itu, fitur yang dihasilkan diringkas menggunakan global average pooling dan diteruskan ke fully connected layer untuk menghasilkan vektor representasi(Tan et al., 2021).

2.2.3. BiLSTM

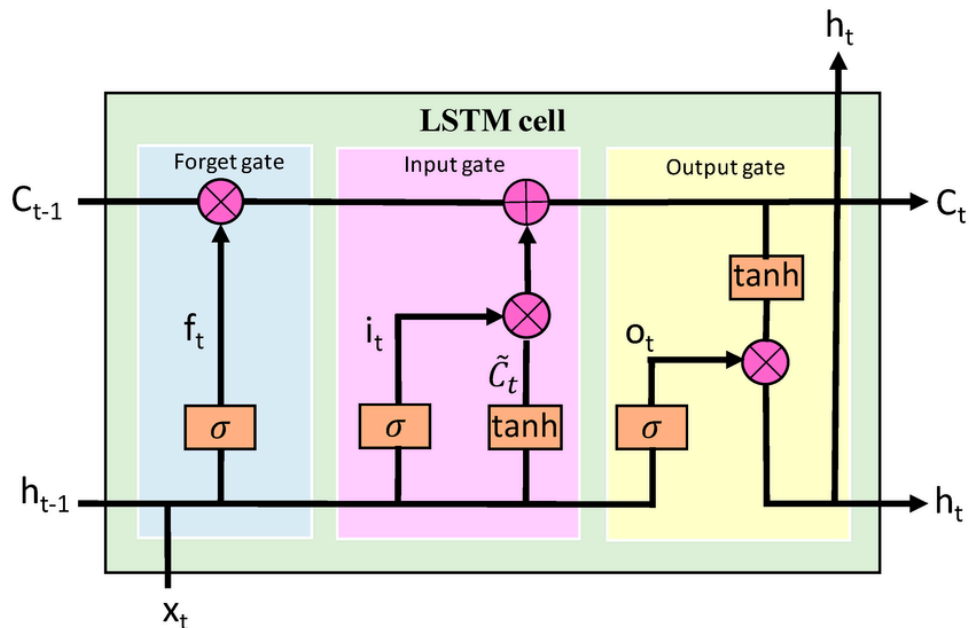
Bidirectional Long Short-Term Memory atau biasa disebut BiLSTM merupakan salah satu jenis dari Recurrent Neural Network yang berhasil mengatasi

keterbatasan dari RNN tradisional. BiLSTM telah dirancang untuk secara efektif mengelola masalah vanishing gradient dan menangkap long term dependencies atau dependensi jangka panjang pada time series. BiLSTM akan melakukan monitoring aliran informasi dari awal ke akhir (maju) untuk memahami konteks berdasarkan informasi masa lalu dan dari akhir ke awal (mundur) untuk menangkap konteks dari informasi masa depan. (Ghosh et al., 2023)



Gambar 4. Arsitektur BiLSTM

Lapisan Bi-LSTM pada gambar 4 akan memproses urutan input dalam arah maju dan mundur secara bersamaan. Selama proses forward, lapisan LSTM menangkap informasi dari masa lalu (langkah waktu sebelumnya), sedangkan selama proses backward, lapisan ini menangkap informasi dari masa depan (langkah waktu berikutnya). Pemrosesan dua arah ini memungkinkan model untuk secara efektif menangkap long-term dependencies dalam input sequences.



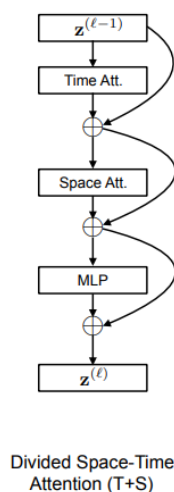
Gambar 5. Arsitektur Lapisan LSTM

Lapisan LSTM sendiri berfungsi seperti biasanya. Model LSTM memiliki fitur sel memori c atau “memory cell c ” yang menyimpan informasi dan menggunakan mekanisme gerbang atau *gate* untuk mengatur aliran informasi. Arsitektur LSTM sendiri terdiri dari input gate, forget gate, output gate, dan status update layer yang masing-masing berfungsi sebagai komponen ANN. Proses dimulai dari *forget gate* yang menggunakan fungsi aktivasi sigmoid (σ) untuk memeriksa input saat ini dan hidden state sebelumnya (H_{t-1}) lalu menentukan informasi mana yang tidak lagi relevan dari long-term memory sebelumnya (C_{t-1}). Selanjutnya, input gate menentukan informasi baru mana yang akan disimpan. Sementara itu, status update layer akan menciptakan kandidat nilai baru untuk long-term memory (C_t). Terakhir, output gate melakukan filter pada long term memory yang sudah diperbarui untuk menghasilkan hidden state atau short-term memory saat ini yang berfungsi sebagai input untuk sequence berikutnya. (Khosravi et al., 2025)

2.2.4. TimeSformer

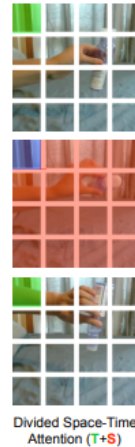
TimeSformer (Time-Space Transformer) adalah arsitektur video classification tanpa konvolusi, yang dibangun sepenuhnya berdasarkan self-attention di domain ruang (spasial) dan waktu (temporal). Model ini memperluas arsitektur Transformer yang digunakan dalam NLP untuk diterapkan pada data video. Model ini diajukan oleh Tim Facebook Research pada sebuah paper yang berjudul “*Is Space-Time Attention All You Need for Video Understanding?*”.

Model ini melihat sebuah video sebagai sebuah urutan *patch* (diambil dari *frame* video, masing-masing berukuran 16x16). Masing-masing *patch* ini kemudian dipetakan secara linear menjadi sebuah *embedding* dan di-augmentasi dengan informasi posisi masing-masing *patch* dalam urutan tersebut. Hal ini memungkinkan untuk *embeddings* tersebut diproses seolah menjadi *token embeddings* yang bisa diserahkan kepada sebuah *Transformer Encoder* dan bisa diproses layaknya sebuah Transformer memproses token yang diambil dari teks pada sebuah tugas NLP. (Bertasius et al., 2021)



Gambar 6. Diagram Alir Desain *Divided Space-Time Attention*.

Blok Transformer ini terdiri dari banyak *attention block* yang ditumpuk secara sekuensial. Jenis desain *attention* yang digunakan dalam implementasi kali ini adalah *Divided Space-Time Attention*, dimana masing-masing embedding akan melalui 2 buah perhitungan *attention* yang berbeda (*Time* dan *Space*) untuk menentukan hubungan temporal dan spasial dari seluruh *embeddings* pada video yang dimasukkan kepada model ini. (Bertasius et al., 2021).



Gambar 7. Visualisasi *Neighborhood* pada *Divided Space-Time Attention*.

Visualisasi *neighborhood* dari desain ini bisa dilihat di gambar 4. Setelah seluruh frame dipecah menjadi *patches*, satu per satu patch akan melewati *time attention* terlebih dahulu (*patch* biru dengan hijau, yaitu *patch* dengan posisi yang sama di frame yang berbeda), setelah itu patch akan melalui *space attention* (*patch* biru dengan merah). (Bertasius et al., 2021)

BAB 3 METODOLOGI

3.1. Dataset yang Digunakan

Penelitian ini memanfaatkan dua dataset utama untuk mendeteksi kekerasan dalam video, yaitu dataset Harvard Dataverse yang berisi skenario rekayasa (*staged scenarios*) dan dataset Kaggle yang mengandung situasi nyata (*real-world scenarios*). Pendekatan kombinasi ini bertujuan untuk menciptakan keseimbangan antara pembelajaran pola kekerasan yang terstruktur dan kemampuan adaptasi model terhadap berbagai kondisi nyata yang kompleks.

Tabel 1 Struktur Dataset yang Digunakan

Sumber Dataset	Jumlah Video	Label		Karakteristik	Sumber Utama
		Violence	Non Violence		
Harvard Dataverse	2.000*	1.000	1.000	Staged video, kualitas rekaman lebih terkontrol	Rekaman dengan <i>staged scenarios</i>
Kaggle (Real Life Violence)	2.000	1.000	1.000	Non-staged, real-world	YouTube (non-staged)
Total	4.000	2.000	2.000		

*Dari total 3.000 video yang tersedia

Dataset Harvard Dataverse terdiri dari video-video yang direkam dengan skenario yang telah dirancang dan diperankan oleh aktor non-profesional. Video-video ini mencakup berbagai konteks kekerasan, seperti kejadian di jalanan, ruang kelas, dan lingkungan rumah tangga. Kualitas rekaman dalam dataset ini relatif terkontrol dengan sudut pandang yang bervariasi, sehingga memberikan representasi yang konsisten untuk pembelajaran pola dasar kekerasan. Dalam penelitian ini, hanya 2.000 dari total 3.000 video yang digunakan agar proporsi dataset ini seimbang dengan dataset real-world dan untuk menghindari bias yang berlebihan terhadap skenario *staged*, yang dapat mengurangi kemampuan generalisasi model.

Di sisi lain, dataset Kaggle Real Life Violence Situations berisi video-video kejadian nyata yang dikumpulkan dari YouTube. Video-video ini menampilkan perkelahian jalanan, konflik spontan, serta aktivitas sehari-hari yang normal seperti berolahraga dan bersosialisasi. Kualitas rekaman dalam dataset ini bersifat amatir dengan penempatan kamera yang tidak terkontrol, mencerminkan kondisi perekaman spontan dalam situasi nyata. Hal ini menyebabkan adanya variasi pencahayaan, sudut pengambilan gambar yang tidak optimal, dan kualitas resolusi yang beragam.

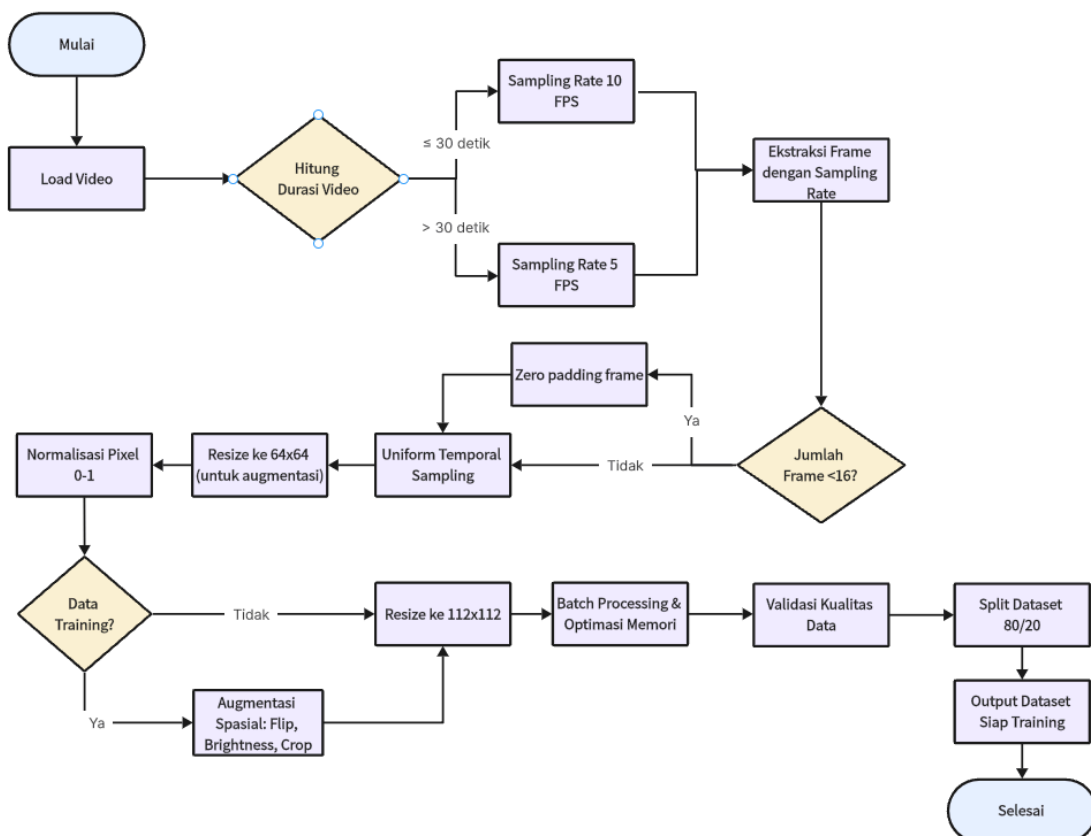
Dengan menggabungkan kedua dataset secara seimbang—masing-masing 2.000 video kekerasan dan 2.000 video non-kekerasan—model dilatih untuk mengenali pola kekerasan

dalam kondisi yang bervariasi. Dataset Harvard memberikan struktur dan konsistensi visual yang membantu model memahami karakteristik dasar tindakan kekerasan. Sementara itu, dataset Kaggle menambahkan keragaman dari situasi nyata dengan kualitas video yang tidak terkontrol. Kombinasi ini diharapkan dapat meningkatkan kemampuan model dalam mengidentifikasi kekerasan dalam berbagai skenario, baik dalam skenario terstruktur maupun dalam kondisi dunia nyata yang lebih kompleks.

3.2. Desain Sistem

3.2.1. Tahap Preprocessing Data

Alur kerja untuk tahap preprocessing bisa dilihat pada *flowchart* berikut:



Gambar 8. Alur Kerja Preprocessing

Proses dimulai dengan ekstraksi frame dari setiap video. Untuk mengakomodasi perbedaan durasi, diterapkan strategi *adaptive sampling rate* dimana video berdurasi hingga 30 detik diekstrak dengan kecepatan 10 FPS, sedangkan video berdurasi lebih dari 30 detik menggunakan 5 FPS. Langkah ini menjaga agar jumlah frame yang diperoleh tetap representatif terhadap keseluruhan isi video.

Setelah frame berhasil diekstrak, dilakukan uniform temporal sampling untuk mendapatkan 16 frame yang terdistribusi merata sepanjang durasi video. Pendekatan ini memastikan representasi temporal yang seimbang. Bila jumlah frame kurang dari 16, maka diterapkan zero-padding dengan menambahkan frame kosong berukuran 64×64 piksel sehingga jumlah minimum terpenuhi. Setiap frame yang diperoleh pada tahap ini

diubah ukurannya ke resolusi awal 64×64 piksel guna menghemat memori sebelum nantinya diubah ke resolusi akhir 112×112 piksel pada tahap akhir preprocessing.

Selanjutnya, seluruh frame dinormalisasi ke rentang piksel 0 hingga 1 untuk mempercepat konvergensi model dan menjaga kestabilan numerik selama proses pelatihan. Normalisasi ini juga berfungsi mereduksi pengaruh pencahayaan yang bervariasi dalam video. Setelah itu, dilakukan augmentasi spasial secara khusus untuk data training, dengan probabilitas 50% per frame. Teknik augmentasi yang digunakan mencakup horizontal flip untuk memperkaya orientasi visual, penyesuaian brightness untuk menyimulasikan kondisi pencahayaan berbeda, serta random crop dan resize untuk menambah variasi komposisi gambar. Augmentasi temporal sengaja tidak diterapkan karena dapat mengacaukan urutan kronologis yang menjadi kunci dalam mendeteksi eskalasi video *violence* yang gerakannya berurutan.

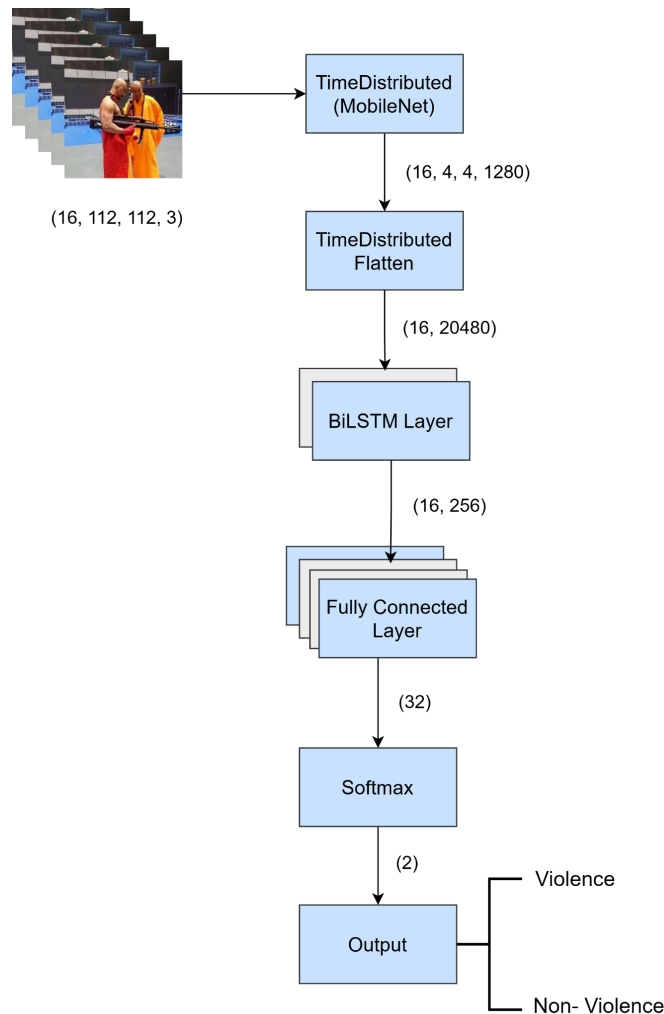
Untuk mendukung efisiensi pemrosesan, seluruh *pipeline* dijalankan dalam batch berisi 32 video. Selain itu, penggunaan tipe data `np.float32` menggantikan `np.float64` berkontribusi signifikan dalam mengurangi beban memori selama proses berlangsung. Pipeline preprocessing ini berbeda antara data training dan testing. Pada data training, dilakukan penyesuaian jumlah frame, augmentasi spasial, normalisasi, dan resize akhir. Sementara itu, data testing hanya mengalami resize dan normalisasi tanpa augmentasi agar hasil evaluasi tetap valid dan bebas dari data *leakage*.

Sebagai penutup, dilakukan validasi kualitas data yang mencakup pengecekan distribusi kelas, durasi video, jumlah frame, ukuran file, serta verifikasi integritas data seperti dimensi array dan rentang nilai piksel. Video yang rusak atau corrupt disaring untuk memastikan tidak ada anomali yang dapat mempengaruhi pelatihan model. Output akhir dari proses preprocessing adalah dataset dalam bentuk array berdimensi $(N, 16, 112, 112, 3)$, di mana N merupakan jumlah video. Label dikodekan secara one-hot untuk kebutuhan klasifikasi biner, dan dataset dibagi menggunakan *stratified sampling* menjadi 80% untuk pelatihan dan 20% untuk pengujian. Tahapan ini memastikan bahwa data masukan ke model sudah dalam format yang optimal, bersih, dan representatif terhadap pola yang ingin dipelajari.

3.2.2. Tahap *Training Model*

3.2.2.1. MobileNet + BiLSTM

Proses training model MobileNetV2 + BiLSTM menggunakan input dari hasil preprocessing yang telah dilakukan. Dapat diketahui bahwa hasil preprocessing pada sub bab sebelumnya, menghasilkan output yang memiliki shape $(N, 16, 112, 112, 3)$ di mana N merupakan batch size. Selanjutnya, pada tahap *build* model, input shape diatur dengan menyesuaikannya seperti hasil preprocessing, yakni 112×112 .



Gambar 9. Design Model MobileNetV2 + BiLSTM

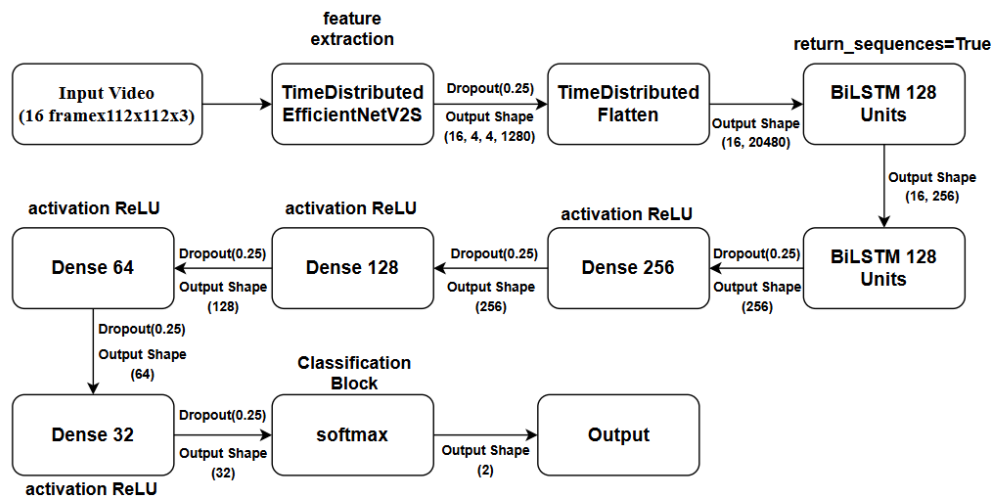
Proses pelatihan dan pengujian model dapat dilihat pada gambar 9. Pertama, input dari hasil preprocessing akan diekstraksi fiturnya menggunakan MobileNetV2 yang dikemas dalam TimeDistributed untuk menangani setiap frame dari input. Dengan ini, sifat sequential masih bisa dipertahankan. Selanjutnya, hasil feature extraction dari MobileNet yang berupa *feature map* ini kemudian diratakan (flattened) menjadi sebuah vektor yang panjang, menghasilkan 16 vektor fitur yang mewakili setiap frame.

Proses berlanjut pada layer BiLSTM. Di sini, model akan menganalisis seluruh urutan frame secara temporal dengan memproses secara forward dan backward. BiLSTM mampu menangkap konteks, pola gerakan, dan hubungan dinamis antar frame yang menjadi penanda ada atau tidaknya tindakan violence. Informasi temporal selanjutnya akan masuk ke dalam Fully Connected Layer (Dense) yang berfungsi untuk memetakan pola-pola kompleks yang telah dipelajari. Pada tahap akhir, lapisan output dengan fungsi aktivasi softmax akan menghitung probabilitas untuk setiap kategori, dan akhirnya menghasilkan keputusan klasifikasi akhir.

3.2.2.2. EfficientNet + BiLSTM

Setiap video diproses menjadi urutan frame berdimensi tetap dengan panjang sequence 16, menjadikan setiap input video direpresentasikan oleh 16 frame. Untuk ukuran setiap frame adalah (112x112) piksel dengan 3 channel warna, sehingga menjadikan input akhir ke model berbentuk (batch_size, 16, 112, 112, 3). Setelah mengatur input size dilakukan proses sebagai berikut:

1. Ekstraksi fitur-fitur spasial dengan EfficientNetV2S. Model ini diimplementasikan dengan argumen **include_top=False**, sehingga hanya digunakan sebagai feature extractor tanpa menggunakan klasifikasi akhir. Karena input berupa urutan frame video berdimensi (16, 112, 112, 3), maka modul **TimeDistributed** digunakan untuk membungkus EfficientNetV2S, sehingga model dapat memproses setiap frame secara independen namun tetap mempertahankan struktur sekuensialnya. Output shape dari EfficientNetV2 menggunakan TimeDistributed adalah (16, 4, 4, 1280).
2. Output dari EfficientNetV2S di-flatten secara terpisah untuk setiap frame dengan menggunakan **TimeDistributed(Flatten())**. Menjadikan output spasial menjadi vektor fitur per frame untuk mempermudah pemrosesan secara sekuensial. Output shape dari layer ini adalah (16, 20480).
3. Setelah urutan frame dengan SEQUENCE_LENGTH=16 berubah menjadi deretan vektor fitur, selanjutnya adalah layer BiLSTM dengan 128 unit LSTM dengan return_sequences=True, pada layer ini mengembalikan output vektor untuk setiap time step dalam sequence (16 frame). Output shapenya menjadi (batch_size, 16, 256). BiLSTM kedua return_sequences=False by default dengan 128 unit. Layer ini menerima input dari BiLSTM pertama dan menjadikan seluruh sequence menjadi satu vektor fitur. Output shapenya menjadi (batch_size, 256).
4. Setelah dapat representasi sequential, fitur diproses melalui beberapa layer Dense: 256 -> 128 -> 64 -> 32 neurons dengan ReLU activation di setiap dense. Dropout(0.25) digunakan di setiap sebelum dense sebagai regularisasi untuk mencegah overfitting.



Gambar 10. Design Model EfficientNetV2S + BiLSTM

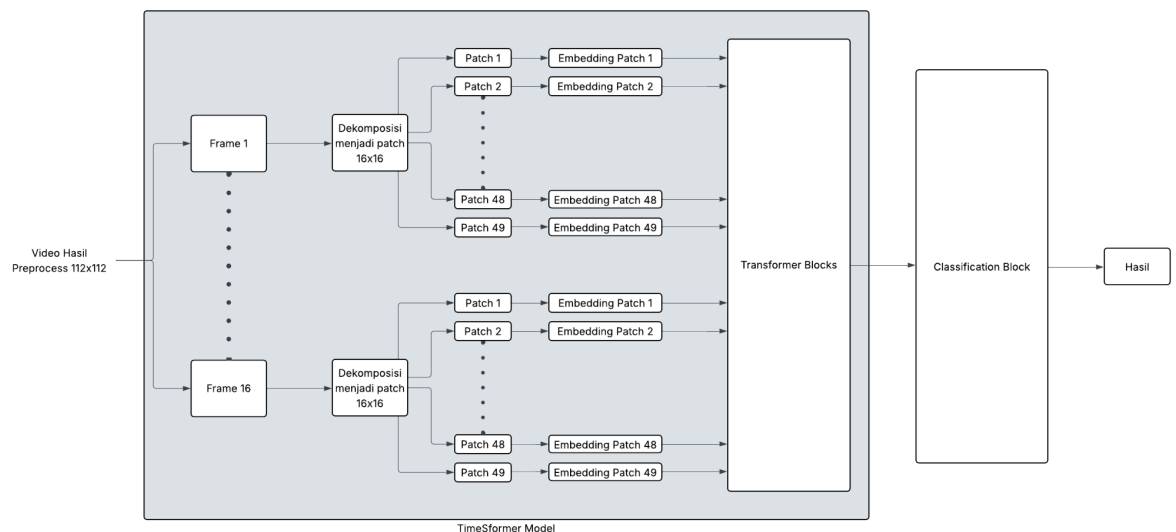
Untuk process training EfficientNetV2S tidak menggunakan weights bawaan yang sudah di train dari imagenet. Seluruh layer pada model EfficientNetV2S dilakukan train ulang untuk mendapatkan weights yang optimal. Hal ini akan menjadikan process backpropagation mengubah semua weights dari awal.

3.2.2.3. TimeSformer

Karena bentuk dan panjang video yang dimasukkan ke dalam model berbeda dengan video-video yang digunakan untuk melatih model saat *pretraining*, maka perlu dilakukan beberapa hal sebagai berikut:

1. Merubah ukuran *positional embeddings* dari $(1 \times 197 \times 768)$ menjadi $(1 \times 50 \times 768)$. Karena satu frame pada video *pretraining* memiliki ukuran 224×224 dan akan menghasilkan 197 patch $(196 + 1 \text{ cls token})$, sedangkan video kami berukuran 112×112 (menghasilkan 50 patch, yaitu $49 + 1 \text{ cls token}$).
2. Merubah ukuran *time embeddings* dari $(1 \times 8 \times 768)$ menjadi $(1 \times 16 \times 768)$. Karena satu klip pada video *pretraining* mengandung 8 frame, sedangkan klip kami mengandung 16 frame.

Perubahan ukuran dilakukan dengan melakukan interpolasi linear terhadap *embeddings* lama sampai ukurannya menjadi sama dengan yang dibutuhkan.



Gambar 11. Diagram Alur Proses Training Model TimeSformer

Proses *training* model TimeSformer dilakukan dengan membekukan *transformer block*, sehingga *weight* bawaan dari hasil *pretraining* dan interpolasi yang dilakukan tidak akan berubah oleh proses *backpropagation*. Proses *backpropagation* hanya akan merubah *weight* dari *classification block* saja. Hal ini dilakukan karena proses *training* ulang keseluruhan blok transformer akan memerlukan sumber daya yang sangat besar, dan tidak mungkin dilakukan dengan sumber daya yang tersedia.

3.2.3. Tahap *Testing* dan Evaluasi

Proses *testing* dan evaluasi model akan dilakukan dengan dataset yang telah dipisahkan sejak awal dan melalui tahap *preprocessing* yang sama dengan dataset *training*. Metrik yang akan dievaluasi adalah akurasi prediksi, serta lama waktu yang diperlukan untuk melakukan proses *training* model hingga proses *testing* selesai. Metrik-metrik ini digunakan untuk menilai efektivitas dan efisiensi model yang telah dibangun.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Gambar 12. Rumus perhitungan *Accuracy*

Evaluasi akan dilakukan terhadap tiga arsitektur model, untuk membandingkan performa dari segi efektivitas (akurasi) dan efisiensi (durasi proses). Model dengan nilai akurasi tertinggi dan waktu eksekusi yang optimal akan dianggap sebagai model terbaik.

BAB 4 HASIL & PEMBAHASAN

4.1. Skenario Pengujian

Pengujian dilakukan dengan menggunakan *Cross Entropy* sebagai fungsi loss pada seluruh skenario. Selain itu, diterapkan teknik *early stopping* dengan *patience* sebesar 10 *epoch* untuk mencegah overfitting. Optimisasi *learning rate* dilakukan menggunakan *ReduceLROnPlateau* dengan faktor penurunan sebesar 0.5, nilai *learning rate* minimum sebesar 0.00005, dan *patience* sebesar 5 *epoch*. Batas maksimal *epoch* diatur sebesar 50 *epoch*.

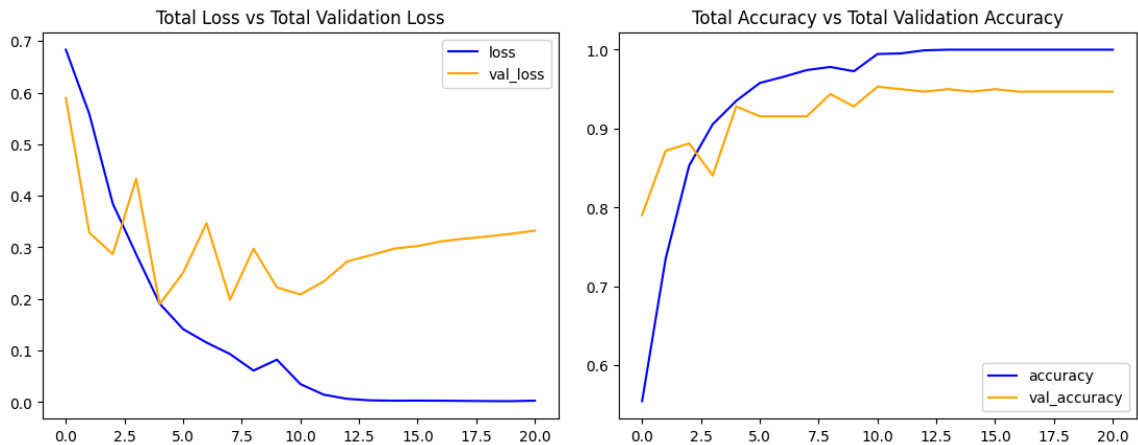
Untuk klasifikasi, akan digunakan *classification block* dengan arsitektur MLP yang terdiri atas lima lapisan berturut-turut dengan jumlah neuron masing-masing 256, 128, 64, 32, dan *output layer* sejumlah 2 (kelas *NonViolence* dan *Violence*). Setiap lapisan akan menggunakan fungsi aktivasi *ReLU* dan *dropout* sebesar 0.25 untuk mengurangi risiko overfitting. Fungsi *dropout* sebesar 0.25 juga akan digunakan pada setiap perpindahan model (dari MobileNet/EfficientNetV2 ke BiLSTM atau Transformer ke *classification block*).

4.1.1. Penggunaan SGD Optimizer

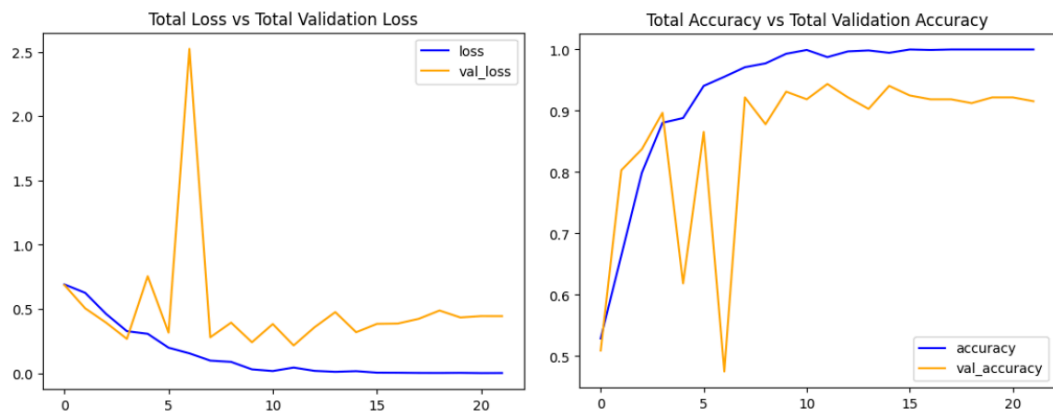
Pengujian dilakukan dengan menggunakan *Stochastic Gradient Descent* (SGD) sebagai *optimizer* pada ketiga model: MobileNet + BiLSTM, EfficientNet + BiLSTM, dan TimeSformer. Dataset yang digunakan pada skenario ini hanya dataset dari Kaggle saja. Tabel berikut menunjukkan hasil akurasi dan total waktu komputasi yang diperlukan dari proses *training* hingga *testing*:

Tabel 2 Hasil Pengujian Skenario Satu

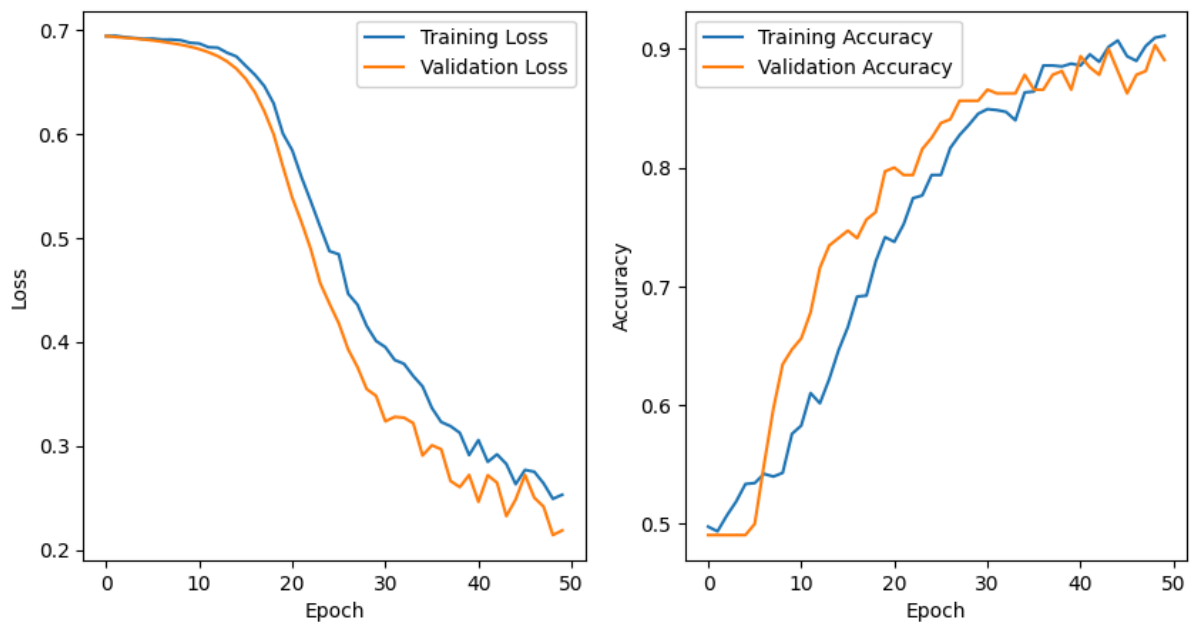
Nama Model	Akurasi <i>Test</i> (%)	Total Waktu (menit)	Jumlah Epochs
MobileNet + BiLSTM	94.64	31	21
EfficientNetV2S + BiLSTM	93.88	85	22
TimeSformer	92.75	47	50



Gambar 13. Perbandingan Loss dan Accuracy MobileNet + BiLSTM.



Gambar 14. Perbandingan Loss dan Accuracy EfficientNetV2S + BiLSTM.



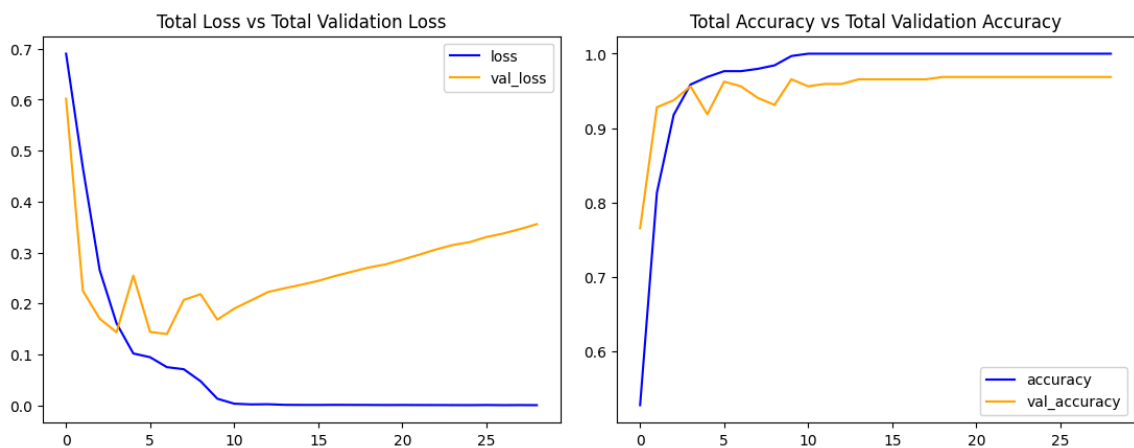
Gambar 15. Perbandingan Loss dan Akurasi TimeSformer.

4.1.2. Penggunaan Adam Optimizer

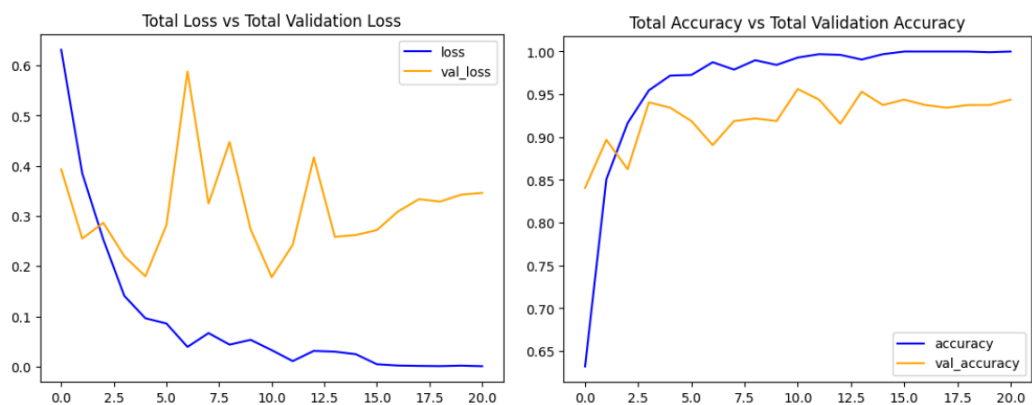
Pengujian dilakukan dengan mengganti *optimizer* yang digunakan menjadi *Adam*, dengan mempertahankan konfigurasi lainnya. Berikut hasil pengujian pada ketiga model:

Tabel 3 Hasil Pengujian Skenario Dua

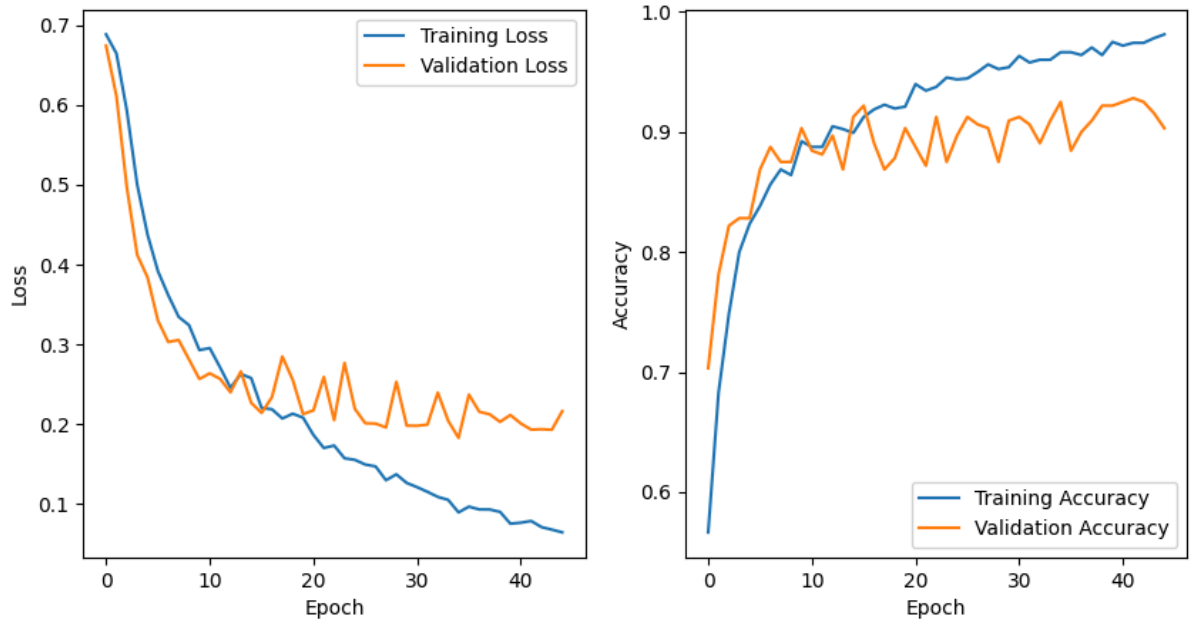
Nama Model	Akurasi <i>Test</i> (%)	Total Waktu (menit)	Jumlah Epochs
MobileNet + BiLSTM	97.47	45	29
EfficientNetV2S + BiLSTM	94.37	87	21
TimeSformer	93.5	43	44



Gambar 16. Perbandingan Loss dan Accuracy MobileNet + BiLSTM.



Gambar 17. Perbandingan Loss dan Accuracy EfficientNetV2S + BiLSTM.



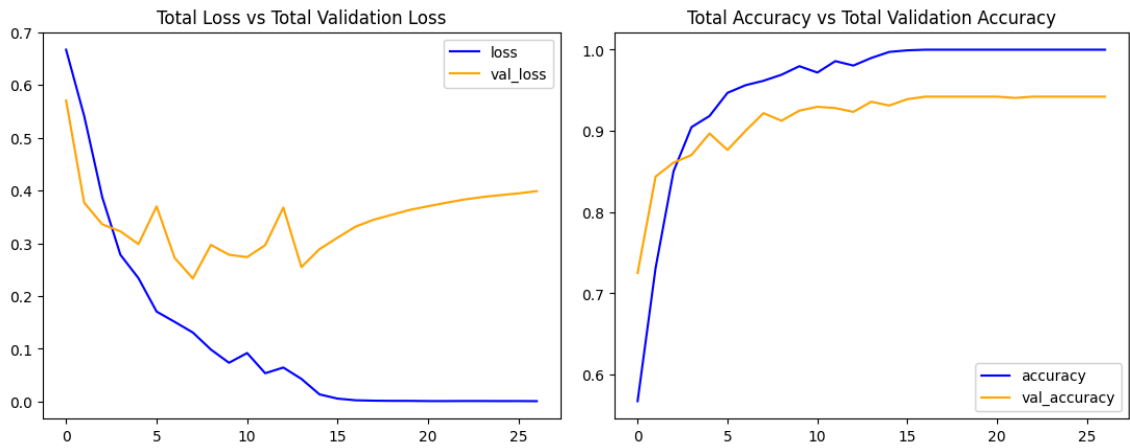
Gambar 18. Perbandingan Loss dan Akurasi TimeSformer.

4.1.3. Penambahan Data dari Sumber Baru

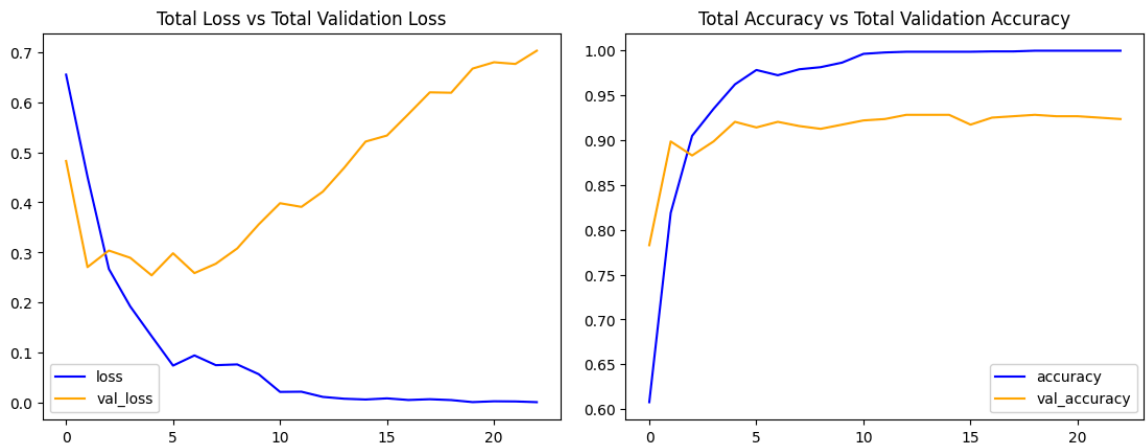
Pengujian ini dilakukan dengan menggabungkan dataset Kaggle dengan dataset dari Harvard Dataverse untuk menguji kemampuan model menerima data lain dengan skenario pengambilan data yang berbeda. Berikut hasil pengujian pada ketiga model:

Tabel 4 Hasil Pengujian Skenario Tiga

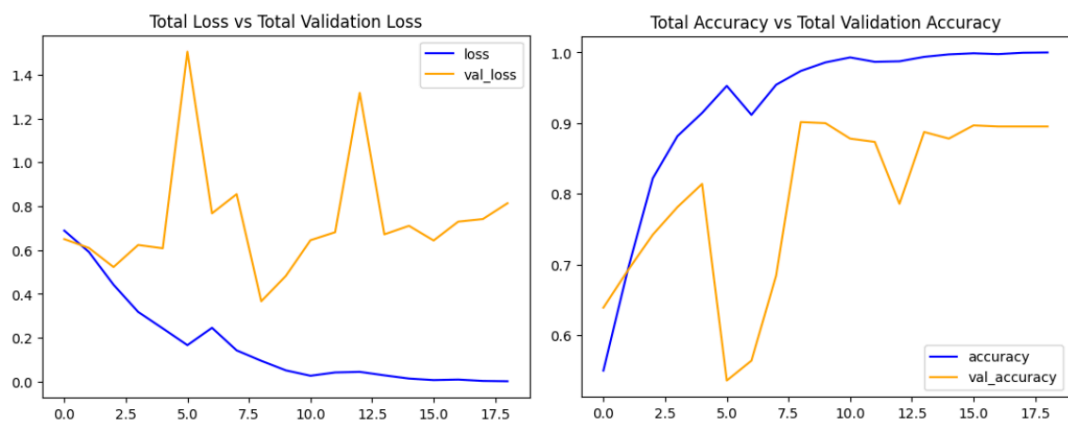
Nama Model	Optimizer	Akurasi <i>Test</i> (%)	Total Waktu (menit)	Jumlah Epochs
MobileNet + BiLSTM	SGD	95.47	72	27
	Adam	93.37	65	23
EfficientNetV2S + BiLSTM	SGD	90.63	133	19
	Adam	91.68	130	18
TimeSformer	SGD	89	95	50
	Adam	91.875	94	49



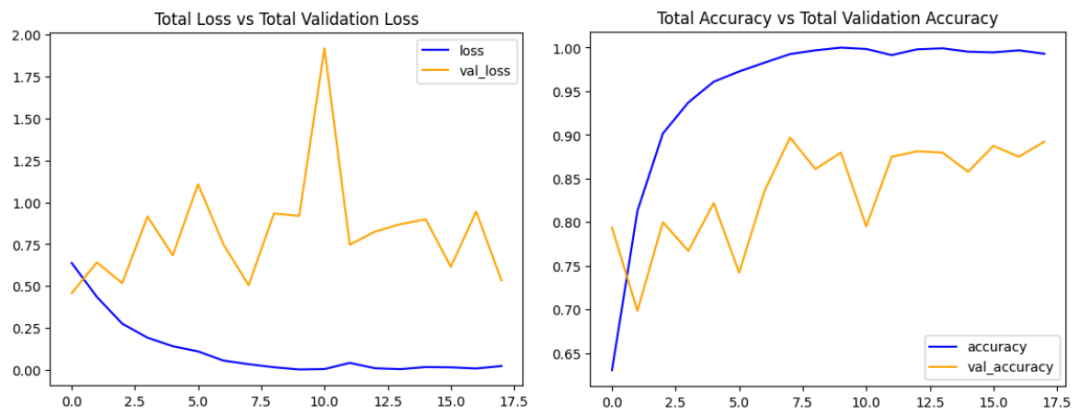
Gambar 19. Perbandingan Loss dan Accuracy MobileNet + BiLSTM optimizer SGD.



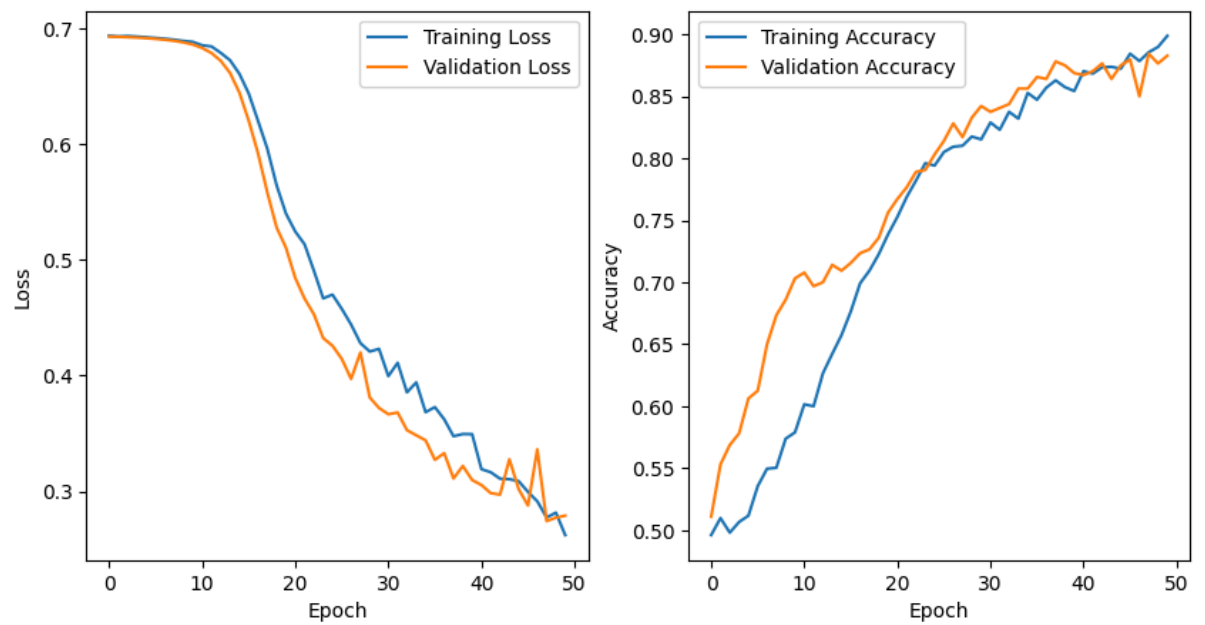
Gambar 20. Perbandingan Loss dan Accuracy MobileNet + BiLSTM optimizer Adam.



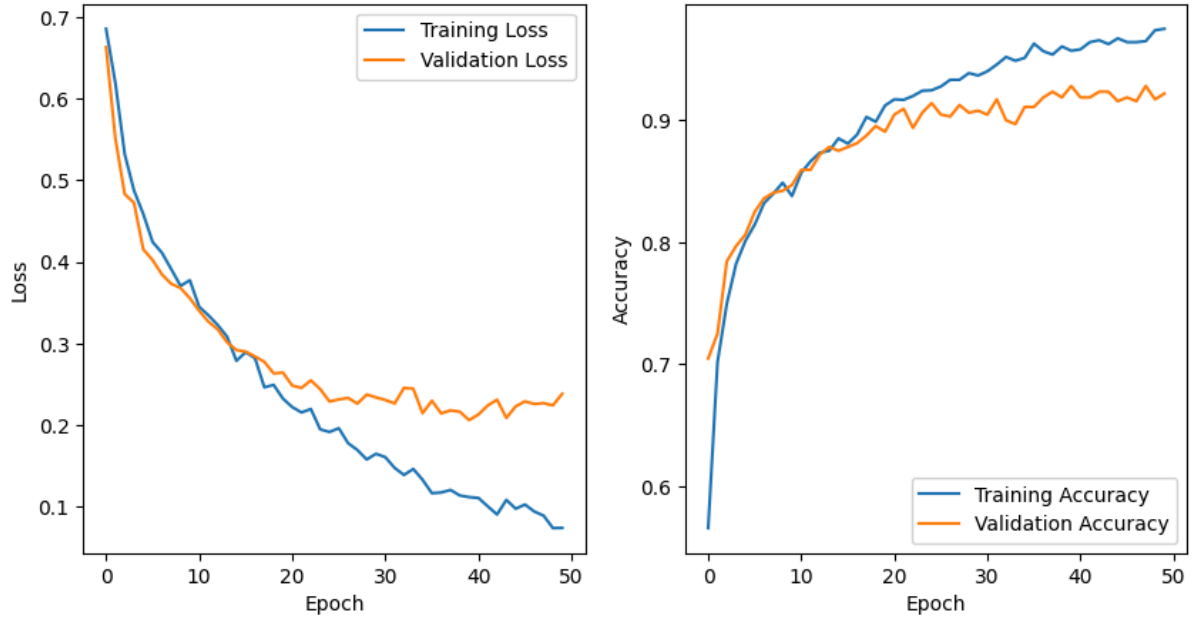
Gambar 21. Perbandingan Loss dan Accuracy EfficientNetV2S + BiLSTM optimizer SGD.



Gambar 22. Perbandingan Loss dan Accuracy EfficientNetV2S + BiLSTM optimizer Adam.



Gambar 23. Perbandingan Loss dan Akurasi TimeSformer Optimizer SGD.



Gambar 24. Perbandingan Loss dan Akurasi TimeSformer Optimizer Adam.

4.2. Pembahasan

Seluruh skenario pengujian telah dilakukan dan didapatkan hasil seperti yang ditampilkan pada tabel 2, 3, dan 4. Pada skenario pertama, dengan konfigurasi *Stochastic Gradient Descent* (SGD) sebagai *optimizer*, diperoleh hasil bahwa model MobileNet + BiLSTM menunjukkan kinerja paling unggul dari segi akurasi dan efisiensi waktu pelatihan, dengan akurasi sebesar 94.64% dan total waktu 31 menit selama 21 *epoch* (sekitar 1.48 menit/*epoch*). Model EfficientNetV2S + BiLSTM mencatatkan akurasi 93.88% dengan waktu 85 menit selama 22 *epoch* (sekitar 3.86 menit/*epoch*), sedangkan TimeSformer mencatatkan akurasi 92.75% dengan waktu 47 menit selama 50 *epoch* (sekitar 0.94 menit/*epoch*). Performa akurasi yang tinggi dari MobileNet yang dikenal ringan dan cepat, dikombinasikan dengan BiLSTM, menunjukkan bahwa model ini dapat secara efisien mengekstraksi fitur spasial dan temporal dari video tanpa memerlukan waktu pelatihan yang panjang. Sementara itu, TimeSformer menunjukkan efisiensi waktu per *epoch* yang sangat baik, namun memerlukan jumlah *epoch* yang jauh lebih banyak untuk mencapai akurasi yang mendekati model-model lainnya.

Pada skenario kedua, dengan konfigurasi Adam sebagai *optimizer*, performa semua model secara umum meningkat dibandingkan dengan penggunaan SGD, khususnya pada MobileNet + BiLSTM yang mencatatkan akurasi tertinggi sebesar 97.47% dalam waktu 45 menit selama 29 *epoch* (sekitar 1.55 menit/*epoch*). Model EfficientNetV2S + BiLSTM memperoleh akurasi 94.37% dengan waktu 87 menit untuk 21 *epoch* (sekitar 4.14 menit/*epoch*), sementara TimeSformer mencatatkan akurasi 93.5% dalam waktu 43 menit selama 44 *epoch* (sekitar 0.98 menit/*epoch*). Peningkatan akurasi yang seragam terjadi pada semua model menunjukkan bahwa *optimizer* Adam sangat efektif untuk meningkatkan performa model. Bahkan untuk model EfficientNet + BiLSTM dan TimeSformer, kedua model ini mampu meningkatkan akurasi mereka dengan jumlah *epoch* yang lebih sedikit dibandingkan dengan menggunakan SGD. Namun terkecuali

TimeSformer, model lainnya memerlukan waktu yang lebih lama dibandingkan skenario sebelumnya untuk menyelesaikan proses *training* model. Bahkan pada model MobileNet+BiLSTM, *training time* mengalami peningkatan hampir sebesar 1.5 kali lipat dibandingkan skenario sebelumnya.

Pada skenario ketiga, semua model diberikan data tambahan untuk menguji kemampuan model belajar pada data baru yang memiliki skenario pengambilan data yang berbeda. Hal menarik yang muncul pada skenario ini adalah sebagian besar model mengalami penurunan akurasi dibandingkan dengan skenario-skenario sebelumnya. Hanya model MobileNet + BiLSTM dengan *optimizer* SGD yang berhasil menaikkan akurasinya menjadi 95.47% selama 27 *epochs* (sekitar 2.67 menit/*epoch*). Banyak model yang mengalami penurunan akurasi karena jenis video yang digunakan oleh sumber baru ini merupakan video rekayasa (*staged*) yang mengakibatkan adanya perubahan pada terjadinya kekerasan di video. Pada data awal, kekerasan terjadi dengan gerakan yang cepat, dan tidak ada sensor yang menutupi muka subjek. Sedangkan pada data tambahan, gerakan rekayasa ini terjadi dengan lebih lambat, serta muka subjek disensor oleh para pembuat data. Kedua hal yang sangat berbeda ini mengakibatkan model-model menjadi kesulitan dalam melakukan klasifikasi kekerasan pada data uji.

BAB 5 KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan hasil pengujian yang telah dilakukan pada tiga model yaitu MobileNet + BiLSTM, EfficientNetV2S + BiLSTM, dan TimeSformer, dapat disimpulkan hal-hal bahwa model MobileNet+BiLSTM merupakan model yang paling unggul secara konsisten di seluruh skenario yang dilakukan, baik dinilai dari segi akurasi maupun *training time* model.

Perubahan *optimizer* yang digunakan dari SGD ke Adam berhasil meningkatkan akurasi semua model secara keseluruhan. Pada model MobileNet + BiLSTM, akurasi berhasil meningkat secara signifikan (hampir 3%). Namun hal ini mengakibatkan naiknya *training time* pada semua model kecuali TimeSformer, yang berhasil meningkatkan akurasi dengan waktu yang lebih singkat.

Penambahan data dari 2000 menjadi 4000 video tidak secara konsisten meningkatkan akurasi. Hanya MobileNet + BiLSTM dengan SGD yang berhasil mempertahankan dan bahkan meningkatkan akurasinya setelah penambahan data. Penurunan akurasi pada sebagian besar model disebabkan oleh karakteristik data tambahan yang berbeda dari data awal, seperti gerakan kekerasan yang lebih lambat dan adanya penyensoran wajah, yang menyebabkan model kesulitan dalam melakukan generalisasi. Hal ini menunjukkan bahwa salah satu keunggulan MobileNet + BiLSTM adalah kemampuan untuk melakukan generalisasi dan beradaptasi pada variasi data.

5.2. Saran

Berdasarkan pengujian yang telah dilakukan, beberapa saran untuk pengembangan dan penelitian lebih lanjut adalah sebagai berikut:

1. Melakukan *Fine-tuning* pada Model TimeSformer: Melihat bahwa TimeSformer memiliki efisiensi waktu per *epoch* yang baik namun akurasi relatif lebih rendah dari model lain, disarankan untuk melakukan *pretraining* lanjutan atau *fine-tuning* pada beberapa *layer* terakhir model ini untuk meningkatkan akurasi.
2. Implementasi pada Sistem Nyata: Disarankan untuk melakukan evaluasi lebih lanjut terhadap model terbaik dalam penelitian ini dengan mengimplementasikannya pada sistem nyata, seperti kamera pengawas (CCTV). Hal ini untuk menguji kemampuan model dalam memberikan klasifikasi yang akurat secara *real-time* atau tanpa *delay* yang signifikan.

DAFTAR PUSTAKA

- Bertasius, G., Wang, H., & Torresani, L. (2021). Is Space-Time Attention All You Need for Video Understanding? (Version 4). *arXiv*. <https://doi.org/10.48550/arXiv.2102.05095>
- Chandola, Y., Virmani, J., Bhadauria, H. S., & Kumar, P. (2021). Lightweight end-to-end Pre-trained CNN-based computer-aided classification system design for chest radiographs. *Deep Learning for Chest Radiographs*, 167–183. <https://doi.org/10.1016/B978-0-323-90184-0.00001-1>
- Fu, Z., Huang, J., & Zhang, L. (2024). *Violence Detection Using EfficientNet-B0 and SA-Separable Convolutional LSTM*. In *2024 International Conference on Neuromorphic Computing (ICNC)*, Chongqing, China. IEEE. <https://doi.org/10.1109/ICNC64304.2024.10987758>
- Garg, D., Verma, G. K., & Singh, A. K. (2024). EEG-based emotion recognition using MobileNet Recurrent Neural Network with time-frequency features. *Applied Soft Computing*, 154, 111338. <https://doi.org/10.1016/J.ASOC.2024.111338>
- Ghosh, S., Ekbal, A., & Bhattacharyya, P. (2023). Natural language processing and sentiment analysis: perspectives from computational intelligence. *Computational Intelligence Applications for Text and Sentiment Data Analysis*, 17–47. <https://doi.org/10.1016/B978-0-32-390535-0.00007-0>
- Khosravi, K., Heddami, S., Jun, C., Bateni, S. M., Kim, D., & Heggy, E. (2025). River total dissolved gas prediction using a hybrid greedy-stepwise feature selection and bidirectional long short-term memory model. *Ecological Informatics*, 90, 103191. <https://doi.org/10.1016/J.ECOINF.2025.103191>
- Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training. (Version 3). *arXiv*. <https://arxiv.org/abs/2104.00298>
- Vijeikis, R., Raudonis, V., & Dervinis, G. (2022). *Efficient Violence Detection in Surveillance*. *Sensors*, 22(6), 2216. <https://doi.org/10.3390/s22062216>
- Vosta, S., & Yow, K.-C. (2022). *A CNN-RNN Combined Structure for Real-World Violence Detection in Surveillance Cameras*. *Applied Sciences*, 12(3), 1021. <https://doi.org/10.3390/app12031021>